

Zero Trust Security Framework for Healthcare Systems

*Project report submitted to the Amrita Vishwa Vidyapeetham in partial
fulfilment of the requirement for the Degree of*

BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING

Submitted by

Sherin Shibu	AM.EN.U4CSE22355
Toufeeq SK	AM.EN.U4CSE22356
Varun Dipu Sankar	AM.EN.U4CSE22359
Harini Sivakumar	AM.EN.U4CSE22363



AMRITA SCHOOL OF COMPUTING
AMRITA VISHWA VIDYAPEETHAM
(Estd. U/S 3 of the UGC Act 1956)
AMRITAPURI CAMPUS
KOLLAM – 690525

MAY 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
AMRITA VISHWA VIDYAPEETHAM
(Estd. U/S 3 of the UGC Act 1956)
Amritapuri Campus
Kollam – 690525



BONAFIDE CERTIFICATE

This is to certify that the project report entitled “Zero Trust Security Framework for Healthcare Systems” submitted by Sherin Shibu (AM.EN.U4CSE22355), Toufee SK (AM.EN.U4CSE22356), Varun Dipu Sankar (AM.EN.U4CSE22359) and Harini Sivakumar (AM.EN.U4CSE22363), in partial fulfillment of the requirements for the award of Degree of Bachelor of Technology in Computer Science and Engineering from Amrita Vishwa Vidyapeetham, is a bonafide record of the work carried out by them under the guidance of **Dr. Swapna M.P** at Amrita School of Computing, Amritapuri during Semester 8 of the academic year 2025–2026.

Guided by

Dr. Swapna M.P
Project Guide

Coordinated by

Sunitha EV
Project Coordinator

Chairperson

Dr. Swaminathan J
Dept. of CSE

Reviewer

Place : Amritapuri
Date : 19 May 2026

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
AMRITA VISHWA VIDYAPEETHAM
(Estd. U/S 3 of the UGC Act 1956)
Amritapuri Campus
Kollam – 690525



DECLARATION

We, **Sherin Shibu** (AM.EN.U4CSE22355), **Toufeeq SK** (AM.EN.U4CSE22356), **Varun Dipu Sankar** (AM.EN.U4CSE22359) and **Harini Sivakumar** (AM.EN.U4CSE22363) hereby declare that this project entitled “**Zero Trust Security Framework for Healthcare Systems**” is a record of the original work done by us under the guidance of **Dr. Swapna M.P**, Centre for CyberSecurity Systems and Network, Amrita Vishwa Vidyapeetham, that this work has not formed the basis for any degree/diploma/associationship/fellowship or similar awards to any candidate in any university to the best of our knowledge.

Place : Amritapuri

Date : 19 May 2026

Signature of the students

Signature of the Project Guide

Acknowledgements

We express our sincere gratitude to everyone who assisted us and provided direction during the course of this project.

Our project guide, **Dr. Swapna M.P**, Center for CyberSecurity Systems and Networks, Amrita Vishwa Vidyapeetham, Amritapuri, provided us with invaluable support through her expert guidance and unending support and her detailed project feedback.

The technical knowledge and assistance from **Hari N.N** and **Prabhakar Krishnan** at the Center for CyberSecurity Systems and Networks should be acknowledged with our thanks.

The research received essential resources and research facilities from the **Head of the Department of Computer Science and Engineering** and faculty members of **Amrita School of Computing, Amritapuri**.

The development of the machine learning model for this project based on the **CSE-CIC-IDS2018 dataset**, which we acknowledge as a vital resource, is underway.

Our families and friends deserve our appreciation for their constant support that provided us with motivation throughout this entire process.

Contents

Contents	v
List of Figures	viii
List of Tables	x
1 Introduction	1
1.1 Motivation	3
1.2 Objectives	3
1.3 Proposed System	4
1.4 Organisation of the Report	4
2 Related Work	5
2.1 Zero Trust Architecture and Microsegmentation	5
2.2 Endpoint Detection and Response in Healthcare	6
2.3 IoT Security and Data Integrity	6
2.4 Summary and Positioning of SecureHealthCare	7
3 Problem Definition	9
3.1 Problem Statement	9
3.2 Scope of the Project	10
3.2.1 Inclusions	10
3.2.2 Exclusions	11
3.3 Novelty of the Proposed Approach	11

4	System Design and Methodology	12
4.1	Experimental Environment	12
4.2	Protected Hospital Environment	13
4.3	Control-Aware Identity and Access Management	14
4.3.1	Multi-Factor Authentication	14
4.3.2	Stateless JWT Authorization	14
4.4	Event Generation and Security-Relevant Activity	15
4.5	Centralized Telemetry and Data Collection	16
4.6	Rule-Based Traffic and Security Analysis	16
4.7	Machine Learning-Based Behavioural Analysis	17
4.8	Centralized Response and Enforcement	18
4.9	Monitoring and Observability	20
5	Proposed System Architecture	21
5.1	Overview	21
5.2	Phase I: Identity and Access Control Plane	21
5.2.1	IAM Server	22
5.2.2	API Gateway, SDP Controller, and SPA Controller	22
5.2.3	Backend Internal Gateway	22
5.2.4	Encryption Service	23
5.2.5	Frontend Application	23
5.3	Phase II: Data Control Plane	23
5.3.1	EDR Agents and Telemetry Collector	23
5.3.2	Rule-Based Detection System	23
5.3.3	ML Engine	24
5.3.4	Automated Response Controller	24
5.3.5	Prometheus and Grafana	24
5.4	Inter-Component Communication	25

5.5	Microsegmentation Model	26
6	Results and Discussion	29
6.1	Validation of SDP Access Control	29
6.2	Validation of Event Generation and Security-Relevant Activity . .	32
6.3	Validation of Centralized Telemetry and Data Collection	33
6.4	Validation of Rule-Based Traffic and Security Analysis	36
6.5	Validation of Machine Learning-Based Behavioural Analysis . . .	37
6.6	Validation of Centralized Response and Enforcement	39
6.7	Monitoring and Observability	40
6.8	Summary of Results	43
7	Conclusion and Future Work	44
7.1	Conclusion	44
7.2	Future Work	45
A	Supplementary Material	46
A.1	Docker Compose Service Overview	46
A.2	CSE-CIC-IDS2018 Dataset Summary	47
A.3	User Roles and Permissions Summary	48
	Bibliography	49

List of Figures

5.1	Architecture of SecureHealthCare	21
6.1	Role-based policy enforcement in the SecureHealthCare SDP demo, showing allow and deny decisions for protected hospital resources	30
6.2	SecureHealthCare SDP demo showing rejection of access to a protected endpoint when valid authentication context is not presented	30
6.3	SPA-based admission in the SecureHealthCare demo, showing successful client validation before protected service access is granted .	31
6.4	Dynamic isolation of the backend-clinical-segment in the SecureHealthCare SDP demo following a security event	31
6.5	Login interface of the hospital portal used to initiate authenticated access	32
6.6	MFA verification stage requiring a six-digit authenticator code before access is granted	32
6.7	Console evidence showing MFA enforcement and issuance of access and refresh tokens after successful verification	33
6.8	Structured ROLE_ACCESS_VIOLATION telemetry generated when a user attempted to access a resource outside the assigned role	33
6.9	Filtered centralized telemetry output showing multiple producers forwarding structured records into a single monitoring collector . .	34
6.10	Compact host-level telemetry record from the centralized collector, showing runtime context contributed by a workstation agent . . .	35

6.11	Structured <code>ROLE_ACCESS_VIOLATION</code> telemetry within the centralized collector, illustrating normalization of role-based security events for downstream analysis.	36
6.12	Injected telemetry record used to simulate suspicious outbound traffic toward a destination IP matched by the exfiltration rule . .	36
6.13	Traffic analyzer output showing deterministic rule-based detection of exfiltration with <code>CRITICAL</code> severity	37
6.14	Final evaluation results of the Random Forest model used for machine learning-based behavioral analysis	38
6.15	Response-controller action history showing severity-based enforcement actions, including <code>revoke_tokens</code> for role violation and <code>isolate</code> for critical exfiltration detection	39
6.16	Recorded isolation and <code>revert_isolation</code> actions demonstrating that centralized enforcement actions are auditable and reversible . . .	40
6.17	Grafana dashboard for the SecureHealthCare SDP and microsegmentation demo, showing observed segments, active segment isolation, registered gateways, telemetry volume, and segment-wise activity trends	41

List of Tables

2.1	Comparison of Existing Approaches and SecureHealthCare	8
5.1	Key Components of SecureHealthCare	25
6.1	Random Forest Model — Final Evaluation Results	38
6.2	Response Actions by Alert Type and Severity	40
6.3	Key Metrics from the SecureHealthCare Grafana Dashboard . . .	42
6.4	Summary of Validation Results	43
A.1	Docker Compose Services in SecureHealthCare Prototype	46
A.2	CSE-CIC-IDS2018 Dataset Statistics	47
A.3	Hospital User Roles and Permitted Resource Endpoints	48

Abstract

The healthcare systems encounter continuous problems because they must safeguard their infrastructure against present cyber threats while keeping patient information and medical services available for continuous use. The security systems which use perimeter protection face difficulties because they trust all entities within their protected area which creates security holes that allow both internal and external attackers to access their network and security systems and internal communication networks.

This report proposes **SecureHealthCare**, a novel Zero Trust security architecture for healthcare settings that integrates IAM, SDP, and EDR solutions. Token-based access control enforces the authentication and authorization procedures, and at the same time, performance monitoring services and endpoints deliver telemetry data to the main system for processing. The detection process includes rule-based and machine learning approaches, which help detect severe threats and trigger automated protection actions like token cancellation, access restrictions, and endpoint isolation.

In order to test the system, a prototype implementation of SecureHealthCare was deployed in a simulated hospital setting using containers. It can be seen that the system implemented identity-based access control and gathered telemetry data from various endpoints.

Besides that, the system detects policy violations and malicious behavior within the network and automatically isolates the source of attack with the help of the response system. Random Forest classifier achieved 95.27% attack recall, confirming the effectiveness of machine learning-based behavioral analysis combined with rule-based detection.

Chapter 1

Introduction

The healthcare industry has experienced its most significant transformation during the past three years through its adoption of electronic health records and medical devices connected to the Internet and cloud-based systems that enable healthcare providers to deliver patient care. The healthcare industry has achieved better results through these technological innovations, but it has also created new risks through new security threats. The increasing deployment of internet-connected devices in healthcare facilities leads to the expansion of potential cyber attack methods. Although IoT devices provide actual benefits to clinical environments their security design falls short because most devices lack protective features which established security methods cannot handle according to [1].

Security systems of organizations depended on the belief that internal network users would be more trustworthy but this assumption proved to be inadequate. Modern threats enter systems through back doors and ransomware attacks succeed because they use methods that remain undetected until an attack occurs. Research shows that harmful attacks emerge from users who have legitimate access to systems. The healthcare sector experiences high risk because security breaches extend beyond an IT issue to include complete hospital shutdowns which result in patient data theft that endangers lives according to [2].

The situation required a complete overhaul of security procedures which resulted in the organization implementing a Zero Trust security framework. Trust

needs to undergo verification, because it should not become an automatic assumption. The execution of Zero Trust in practical scenarios requires more substantial efforts than simply establishing new organizational regulations. Organizations must implement continuous monitoring through advanced tools to detect suspicious activities that will help them identify and isolate those activities while maintaining the protection of their entire network system. Today, security systems utilize machine learning methods to evaluate the reliability of the user and device according to current conditions, while detecting potentially harmful activities before those activities become critical security breaches [4].

Endpoint Detection and Response (EDR) solutions provide essential support for this particular type of security protection. Security teams monitor endpoint devices through continuous observation of operational activities which includes process tracking, file monitoring, network traffic analysis, and system log review. EDR systems automatically take action to control threats when any unusual behavior is detected, preventing further damage [5].

The combination of EDR and network microsegmentation provides organizations with better security protection. Microsegmentation divides the network into smaller and more controlled sections rather than allowing all systems to communicate freely within the same environment. The clinical systems and administrative systems can operate separately which prevents attackers from moving unchecked throughout the network during security breaches. The containment strategy reduces the overall impact of security incidents which healthcare organizations that face constant threats must deal with [6].

1.1 Motivation

The healthcare infrastructure sector faces its highest risk from cyberattacks because attackers seek access to sensitive patient information and they aim to disrupt essential hospital activities. The current threat environment requires organizations to adopt security methods which extend beyond traditional perimeter-based security systems since they need protection against external attackers and internal trusted employees. SecureHealthCare aims to develop an operational security framework which protects all components of healthcare architecture through its implementation of Zero Trust security principles.

1.2 Objectives

The key objectives of this project are:

1. To deploy Endpoint Detection and Response (EDR) solutions to improve threat detection across critical healthcare systems supporting a Zero Trust approach.
2. To apply microsegmentation to control access between systems, reduce lateral movement, and strengthen overall security.
3. To install endpoint monitoring agents on critical systems for real-time visibility into processes, files, network activity, and system logs.
4. To centralize security monitoring and threat detection by collecting and analyzing endpoint and network events using a combination of rule-based detection and machine learning-based anomaly analysis.
5. To demonstrate improved threat response through automated actions such

as access control enforcement and network microsegmentation during simulated ransomware and insider attacks.

1.3 Proposed System

This report presents SecureHealthCare, a healthcare security solution which implements Zero Trust architecture through its combination of Endpoint Detection and Response (EDR) and network microsegmentation to safeguard system endpoints and their internal network traffic. The EDR system functions through agents which send gathered data to a main analytical system that detects security threats by using its hybrid detection system which combines rule-based analysis with machine learning-based anomaly detection. The system prototype detects policy violations and suspicious network activity in real time while automatically containing threats through network isolation.

1.4 Organisation of the Report

The remainder of this report is structured as follows. Chapter 2 reviews the existing literature on Zero Trust Architecture, network microsegmentation, and EDR systems in healthcare. Chapter 3 presents the problem definition and the scope of the project. Chapter 4 describes the system design and methodology, detailing the deployment of EDR agents, network policies, and hybrid threat detection. Chapter 5 describes the proposed system architecture in detail. Chapter 6 presents experimental results, detection performance, and practical deployment considerations. Chapter 7 concludes the report and outlines the directions for future work.

Chapter 2

Related Work

Healthcare digital transformation has changed how clinical care works and how hospitals operate and how patients access their medical records. The present transformation proceeds through three linked platforms which include Internet of Medical Things (IoMT) devices and cloud-based Electronic Health Records (EHR) systems and mobile health applications that support remote patient monitoring and telemedicine [8]. New technologies make it possible to achieve better diagnoses and higher operational efficiency which leads to better clinical outcomes but these technologies introduce new security vulnerabilities. Advanced persistent threats (APTs) now identify the healthcare industry as their primary target as attackers seek to steal protected health information and launch ransomware attacks that disrupt essential operations [9].

2.1 Zero Trust Architecture and Microsegmentation

The traditional security model contains security vulnerabilities that Zero Trust Architecture effectively removes through its requirement of authenticating all users and devices that attempt to access the network from any geographical location. Software-Defined Perimeters (SDP) and microsegmentation technologies provide the essential components for this system. A ZTA system using SDP with

VPNs for authentication verifies users before they gain access to resources that remain hidden from unauthorized users who attempt to scan the network to access hospital databases [10]. A ZTA design pattern employs microsegmentation with cybersecurity domains to isolate workloads [11], which matches simulation results that demonstrate attacker movement becomes restricted through detailed network segmentation [6]. New policy automation systems solve the problems that manual segmentation creates when used in clinical environments that require continuous changes.

2.2 Endpoint Detection and Response in Healthcare

ZTA controls resource access, while Endpoint Detection and Response (EDR) systems protect devices by detecting unauthorized threats and identifying ransomware attacks which threaten healthcare operations. The system enables automatic security measures to function in various devices, including medical health monitoring systems. A Deep Learning-based ransomware detection system uses LSTM networks to analyze API call sequences for early crypto-ransomware identification [12]. LSTM models outperform existing approaches in identifying complex variants of Android malware that operate on mobile health devices [13]. The agent-based system known as NextEDR combines cloud-based intelligence with on-site processing to deliver immediate automated defense capabilities [5].

2.3 IoT Security and Data Integrity

IoMT devices create new security risks because their security protections are inadequate. Previous research has addressed these security threats through multiple

methods. The implementation of Snort-based intrusion detection systems provides rule-based intrusion detection which secures IoT devices against known attacks that target devices with limited resources [7]. LSTM-based models enhance machine learning capabilities to identify advanced threats that exist throughout all IoMT endpoints [14]. The hybrid security approach combines rule-based detection with ML-based methods to create a system capable of detecting intrusions across IoT networks [15].

2.4 Summary and Positioning of SecureHealth-Care

SecureHealthCare develops an integrated security system through the combination of Zero Trust access controls, network segmentation, and EDR machine learning detection together with rule-based detection. The system combines all existing security challenges to solve problems that previous security methods treated as separate issues that needed different solutions through VPN access control and independent ransomware detection and basic IoT monitoring. The system automatically isolates the affected host when it detects threats to the endpoint while simultaneously blocking network access to prevent further system damage.

The integrated approach solves essential security problems because network segmentation can block threats to authorized devices only as a single defence method which fails to identify advanced attacks and dedicated IoT monitoring systems lack the capability to track endpoint network connections. SecureHealth-Care delivers a scalable defense system that protects healthcare environments against ransomware attacks and other cybersecurity threats through its combination of real-time threat detection and automated network isolation. Table 2.1

provides a comparative summary of the existing approaches and how SecureHealthCare advances upon them.

Table 2.1: Comparison of Existing Approaches and SecureHealthCare

Approach	Access Control	Threat Detection	Automated Re- sponse
SDP over VPN [10]	SDP + VPN	None	None
Standalone Ransomware Detection [12]	None	LSTM (ransomware)	None
Basic IoT Monitoring [7]	Rule-based IDS	Snort signatures	None
NextEDR [5]	None	Agent-based EDR	Automated mitigation
SecureHealthCare (Proposed)	IAM + SDP + JWT	Rule-based + Random Forest ML	Token revocation, isolation, IP block- ing

Chapter 3

Problem Definition

3.1 Problem Statement

Healthcare security systems that use traditional perimeter-based protection methods base their security model on the assumption that all network users and devices present in the system possess complete trustworthiness. This assumption creates a major security weakness that affects contemporary healthcare systems that experience increasing risks from both external attacks and internal security breaches. The systems do not deliver necessary functions which include user and device authentication checks and network asset tracking and security breach alerts.

The healthcare sector is particularly vulnerable because:

- Electronic Health Records (EHRs), medical IoT devices, and administrative systems coexist within shared network environments with inadequate segmentation and isolation.
- Security breaches compromise sensitive patient data, undermining privacy, confidentiality, and trust in healthcare institutions.
- Continuous monitoring and traceability create accountability mechanisms that allow detection and analysis of malicious activities.
- Existing security solutions are fragmented and address isolated threats,

lacking an integrated, auditable, and adaptive defense mechanism suitable for dynamic healthcare settings.

The existing security system delivers insufficient protection because it requires an advanced security framework which provides continuous user trust evaluation and comprehensive access control and centralized security operations monitoring with traceable and auditable functions.

This work aligns with **SDG 16: Peace, Justice and Strong Institutions**, as it strengthens accountability, transparency, and trust in healthcare systems by enabling secure access control, continuous monitoring, and audit-driven security enforcement.

3.2 Scope of the Project

3.2.1 Inclusions

1. Implement Endpoint Detection and Response (EDR) integrated with network microsegmentation to enhance Zero Trust security in healthcare systems.
2. Deploy endpoint monitoring agents on critical systems to collect real-time data on processes, files, network activity, and logs.
3. Enable centralized correlation of security events using a combination of rule-based detection and machine learning-based anomaly analysis.
4. Demonstrate EDR and microsegmentation's role in Zero Trust through automated responses such as access revocation and network isolation.
5. Simulate attack scenarios to validate threat detection and mitigation workflows.

3.2.2 Exclusions

1. Deployment of EDR agents or microsegmentation policies in live healthcare networks.
2. Integration with actual hospital endpoints, patient databases, or proprietary medical systems.
3. Use of commercial EDR platforms or enterprise-grade network segmentation tools.

3.3 Novelty of the Proposed Approach

The project introduces a new Zero Trust framework for healthcare that establishes security through Endpoint Detection and Response and microsegmentation to protect both network paths and endpoints. The system uses real-time telemetry and hybrid threat detection together with automated isolation to stop ransomware attacks and insider threats from spreading. The framework delivers adaptive security solutions for healthcare systems through simulated workflows that show its capacity to deal with evolving security threats. The key novelties are as follows:

- Integration of continuous endpoint monitoring and network isolation into a single Zero Trust healthcare system.
- Use of centralized analysis and automated responses to contain threats and protect sensitive healthcare systems.
- A two-phase architecture that separates authentication and authorization (Phase I) from monitoring, detection, and response (Phase II) while keeping them tightly coupled.

Chapter 4

System Design and Methodology

The SecureHealthCare framework was developed as a two-phase Zero Trust architecture to protect healthcare applications and endpoints. The first phase establishes authentication and authorization systems together with SDP-driven access control and data encryption methods, whereas the second phase handles telemetry gathering and threat detection through rule-based and machine learning methods, which it follows with automated response procedures that include access restrictions and token cancellation and system lockdown. These two phases work together to establish verification methods for access requests while tracking system operations and identifying abnormal activities, which leads to protective measures when necessary.

4.1 Experimental Environment

The framework was deployed in a testing environment using Docker Compose as its local implementation platform. The setup included:

- **Core services:** frontend, backend, IAM, API gateway, SDP controller, encryption service, monitoring service, response controller, PostgreSQL database, and ML engine.
- **Monitoring components:** Prometheus for metric collection and Grafana for dashboard visualization.

- **User roles:** doctor, nurse, receptionist, accountant, pharmacist, administrator, lab technician, and patient.
- **ML training data:** The CSE-CIC-IDS2018 dataset for training the Random Forest model used in behavioral analysis.

The environment enabled testing of access control, telemetry collection, threat detection, automated response, and monitoring systems under conditions simulating actual service usage.

4.2 Protected Hospital Environment

The architectural design centers on hospital services which consist of eight components that include frontend, backend API, IAM service, database, encryption service, monitoring service, ML engine, and response controller. The system consists of multiple services which function independently without establishing direct communication between them. The system employs Software-Defined Perimeter (SDP) to control inter-service access by applying identity-aware access control together with network segmentation and explicit trust boundaries.

The API Gateway protects requests by validating the bearer token before passing the request path and method to the SDP Controller. The controller evaluates the request against predefined policy rules based on the user's role, permissions, and requested resource. The system forwards only permitted requests to the target service while it denies access to all other requests.

4.3 Control-Aware Identity and Access Management

The first phase of the methodology focuses on eliminating partial trust through a robust Identity and Access Management (IAM) system. The system grants users access after they successfully complete both cryptographic identity verification and multi-factor authentication.

4.3.1 Multi-Factor Authentication

To prevent unauthorized access from compromised credentials, a Time-based One-Time Password (TOTP) mechanism is implemented. For any user u , the verification of the 6-digit code C_t at time t is defined as:

$$V(u, t) = \begin{cases} \text{True,} & \text{if } C_t = \text{TOTP}(K_u, \lfloor t/\Delta t \rfloor) \\ \text{False,} & \text{otherwise} \end{cases} \quad (4.1)$$

where K_u is the unique 32-byte secret key and Δt is the 30-second time step. The system allows a window of ± 1 to account for network latency. Passwords are verified using bcrypt-based hashes, followed by issuance of a short-lived JWT access token and a longer-lived refresh token; login is completed only after TOTP verification using an authenticator application such as Microsoft Authenticator.

4.3.2 Stateless JWT Authorization

Upon identity verification, the IAM server issues a JSON Web Token (JWT) containing granular permissions. The integrity of the claims is secured via HMAC-SHA256:

$$\sigma = \text{HMAC-SHA256}(m, k) \quad (4.2)$$

where m represents the header and payload (user role, department, and expiration), and k is the server-side secret. This token must be presented in the authorization header of every API request to the healthcare infrastructure.

4.4 Event Generation and Security-Relevant Activity

Event creation within the designed system is determined by authorized user activity from eight roles of hospital staff: admin, doctor, nurse, receptionist, laboratory specialist, pharmacist, accountant, and patient. After authorization, normal and suspicious events are created through role-constrained actions. The doctors' endpoints include patients', appointments, laboratories', and record-related endpoints, while accountants can perform accounting and have only limited access to patients. Any attempt by the user to access the resource not assigned to the particular role leads to an event creation, which is labeled as `ROLE_ACCESS_VIOLATION`. This event contains structured data including the user's role and email, request method, and the path used.

The system creates security events through its file protection system which encrypts and decrypts sensitive files and report data using AES-256-GCM encryption. The monitoring services track user login attempts and MFA outcomes and encryption activities and file operations and multiple decryption attempts which produce two types of evidence for hospital behavior that will be used to detect and address security incidents.

4.5 Centralized Telemetry and Data Collection

Security-relevant activities from multiple producers are transformed into structured JSON telemetry which is then forwarded to a single monitoring collector to establish centralized data collection and telemetry systems. The following producers generate data for this system:

- The role-aware middleware associated with the backend and IAM services produces request-level telemetry information, including the identity and role of the user, HTTP method, request path, status code, request duration, and IP address.
- The backend security monitor reports on the occurrences of authentication events, among others, like failed login attempts.
- A telemetry agent producing host-level telemetry information includes host id, timestamp, role identity, process activity, file activity, network activity, system logs, database activity (optional), and alerts.

The collector assigns each telemetry record a `receivedAt` timestamp which it saves to a telemetry log and keeps in its in-memory `recentTelemetry` buffer while making the data available through its telemetry endpoint for upcoming analysis.

4.6 Rule-Based Traffic and Security Analysis

Once telemetry has been centralized, the traffic analyzer continuously retrieves recent telemetry from the collector and evaluates each item against a fixed set of code-defined detection rules. The implemented rule set includes:

- **Exfiltration detection:** classifies telemetry as `EXFILTRATION` with `CRITICAL` severity when the destination matches one of the suspicious IPs hard-coded in the analyzer: `8.8.8.8`, `1.1.1.1`, or `185.220.101.1`.
- **Role-based access violation detection:** classifies telemetry as `ROLE_ACCESS_VIOLATION` with `HIGH` severity when `eventType = ROLE_ACCESS_VIOLATION`.
- **Ransomware-style file detection:** classifies telemetry as `RANSOMWARE_INDICATOR` with `CRITICAL` severity when suspicious file events use extensions such as `.encrypted`, `.lock`, `.ransom`, `.crypto`, or `.locked`.
- **Brute-force authentication detection:** classifies telemetry as `BRUTE_FORCE_ATTEMPT` with `HIGH` severity when authentication-related failures from the same host reach five within a fifteen-minute window.
- **Database activity detection:** converts database alerts including `DB_UNAUTHORIZED_TABLE_ACCESS`, `DB_MASS_DELETE`, `DB_BULK_DATA_READ`, and `DB_SCHEMA_CHANGE` into rule-based database alerts.

Before sending alerts, the analyzer applies deduplication to avoid repeated alert flooding.

4.7 Machine Learning-Based Behavioural Analysis

Machine learning-based behavior analysis works together with the rule engine because the traffic analyzer sends all telemetry data to the ML service through the `/predict` endpoint. The ML service operates the Random Forest model

together with its feature configuration because it processes telemetry data into numerical features which track network behavior and process activity and file usage and log data and database operations. The system evaluates the scaled features against the established threshold to identify whether an event exhibits anomalous behavior.

The model produces a structured output which includes the following elements: `anomaly_score`, `is_anomaly`, `classification`, `action`, `confidence`, identity fields, and model-specific outputs. The analyzer creates an `ML_ANOMALY` alert when it detects an anomalous result which it sends to the response layer.

4.8 Centralized Response and Enforcement

The response controller receives alerts through the `/alert` endpoint and acts as the enforcement layer of the architecture. The controller supports multiple enforcement paths:

- `ROLE_ACCESS_VIOLATION` events trigger immediate token revocation through the IAM service.
- `CRITICAL` alerts result in `isolate` actions and may also trigger token revocation and IP blocking.
- `HIGH` severity alerts produce `restrict` actions with IAM-based controls when relevant.
- Non-critical ML detections are recorded as `log_only`.

All response actions are persistent for audit and recovery. The controller writes security logs to disk, stores and exposes action history, and supports roll-back by recording a `revert_isolation` entry for the affected host.

The automated response loop is summarized in Algorithm 1.

Algorithm 1 SecureHealthCare: Automated Response Loop

Require: Telemetry Stream S , Rule Set R

Ensure: System State $\in \{\text{Secure}, \text{Isolated}\}$

```
1: while System Operational do
2:   Receive Telemetry  $S$  from EDR Agent
3:   if Match( $S, R$ ) = True then
4:     Generate Alert with Severity  $\sigma$ 
5:     if  $\sigma = \text{CRITICAL}$  then
6:       Identify Host ID and Network Segment
7:       Trigger Isolation Policy (Response Controller)
8:       Update Network Adjacency Matrix  $A_{ij} = 0$ 
9:       return Isolated
10:    else if  $\sigma = \text{HIGH}$  then
11:      Trigger Token Revocation via IAM
12:      Restrict network access
13:    else
14:      Record as log_only
15:    end if
16:  else
17:    Maintain Secure State
18:  end if
19: end while
```

4.9 Monitoring and Observability

The deployed architecture establishes real-time system monitoring through its tracking system which delivers operational and security performance data in Prometheus format. The metrics exporter maintains counters for allowed connections, blocked attempts, security violations, isolation test results, overall security score, and telemetry volume. Prometheus is configured to scrape the monitoring service together with the backend, database, encryption, frontend, and IAM services. The Grafana dashboard system provides prebuilt panels which display security score metrics and associated monitoring data.

Chapter 5

Proposed System Architecture

5.1 Overview

The SecureHealthCare framework implements a multi-layered Zero Trust Architecture (ZTA) tailored for healthcare environments. As illustrated in Figure below, the architecture is divided into two primary planes: the **Identity and Policy Plane (Phase I)**, which handles continuous authentication and authorization, and the **Data Plane (Phase II)**, which executes endpoint monitoring and automated network isolation.

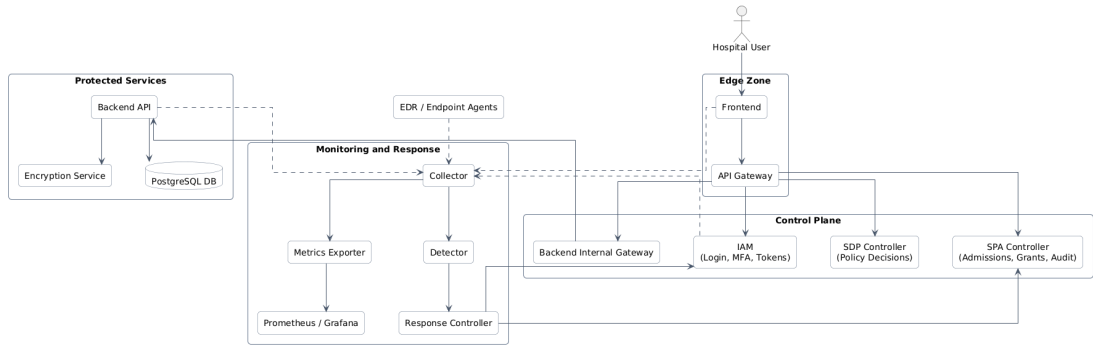


Figure 5.1: Architecture of SecureHealthCare

5.2 Phase I: Identity and Access Control Plane

Phase I is responsible for ensuring that no user or device is granted access without continuous verification. The key components of Phase I are:

5.2.1 IAM Server

The Identity and Access Management (IAM) server handles registration, login, MFA verification, and token issuance. It is the authoritative source of identity within the framework. All authentication events are logged for audit and telemetry purposes.

5.2.2 API Gateway, SDP Controller, and SPA Controller

The API Gateway acts as the public entry point for all client requests. It validates bearer tokens and forwards request metadata, such as the requested path and method, to the SDP Controller for policy evaluation. The SDP Controller checks whether the user role, permissions, and requested resource satisfy the configured access rules.

To strengthen service exposure control, the framework also uses a SPA (Single Packet Authorization) Controller, which issues temporary admissions and short-lived grants only to validated clients. This ensures that protected services are not directly exposed and can be reached only through authorized gateway chains. Together, the API Gateway, SDP Controller, and SPA Controller implement identity-aware and policy-driven access enforcement.

5.2.3 Backend Internal Gateway

The internal backend gateway protects the clinical application segment from direct exposure. Even after a request is accepted by the external API gateway, it must still cross the internal gateway before reaching the backend services. This creates an additional enforcement point between the control plane and the protected backend segment, ensuring that sensitive healthcare services remain accessible only through explicitly approved paths.

5.2.4 Encryption Service

Sensitive data, including patient records and file uploads, is encrypted using AES-256-GCM before being stored in the database. Decryption is only possible through the encryption service, which also logs all encryption and decryption operations as security-relevant events.

5.2.5 Frontend Application

The frontend presents the hospital portal to users and enforces the multi-stage authentication workflow, including credential verification followed by TOTP-based MFA.

5.3 Phase II: Data Control Plane

Phase II is responsible for continuous monitoring, threat detection, and automated response. It consists of:

5.3.1 EDR Agents and Telemetry Collector

EDR agents are deployed on critical endpoints, such as hospital laptops, administrative workstations, and IoT devices. Each agent collects real-time telemetry, including process states, file activity, network connections, and system logs. This data is pushed to the central telemetry collector at regular intervals.

5.3.2 Rule-Based Detection System

The rule-based detection engine evaluates incoming telemetry against a library of healthcare-specific threat signatures. When a rule matches, the engine generates

a structured alert containing severity level, event type, host identity, timestamp, and contextual details.

5.3.3 ML Engine

The ML engine runs in parallel with the rule engine. It accepts telemetry items through the `/predict` endpoint, transforms them into feature vectors, and evaluates them against the trained Random Forest model. Anomalous events are converted into `ML_ANOMALY` alerts and forwarded to the response layer.

5.3.4 Automated Response Controller

The response controller functions as the centralized enforcement layer of Phase II. It receives alerts from both the rule engine and the ML engine, determines the appropriate response based on alert type and severity, and executes actions such as token revocation, access restriction, segment isolation, and rollback tracking. All actions are persistently recorded to provide a complete audit trail of containment decisions.

5.3.5 Prometheus and Grafana

Prometheus and Grafana provide the observability layer for the SecureHealthCare framework. Prometheus collects operational and security-related metrics from multiple services, while Grafana visualizes these metrics through dashboards that expose trends in connectivity, blocked attempts, telemetry volume, security score, and overall system behavior. This monitoring layer supports continuous visibility into both normal operation and active security events.

5.4 Inter-Component Communication

The SecureHealthCare architecture follows a controlled communication model in which services interact only through explicitly approved paths. Client requests originate at the frontend, traverse the API gateway, are evaluated by the SDP and SPA control components, and are then forwarded through the backend internal gateway before reaching the protected backend service. The backend alone is allowed to communicate with highly sensitive components such as the database and encryption service. In parallel, endpoints and services forward telemetry to the centralized collector, which supplies data to the rule engine, ML engine, and monitoring stack.

Table 5.1 summarizes the principal architectural components and their roles.

Table 5.1: Key Components of SecureHealthCare

Component	Phase	Role
IAM Server	Phase I	Authentication, MFA, token issuance
API Gateway	Phase I	External request entry, token validation, request forwarding
SDP Controller	Phase I	Policy-based authorization decisions
SPA Controller	Phase I	Client admission, grant issuance, access mediation
Backend Internal Gateway	Phase I	Protected path enforcement into backend services
Encryption Service	Phase I	AES-256-GCM data protection
Frontend Application	Phase I	User interaction and MFA workflow
EDR Agents	Phase II	Endpoint telemetry collection

Continued on next page...

Component	Phase	Role
Telemetry Collector	Phase II	Centralized telemetry aggregation
Rule Engine	Phase II	Deterministic threat detection
ML Engine	Phase II	Behavioural anomaly detection
Response Controller	Phase II	Automated enforcement and audit logging
Prometheus / Grafana	Phase II	Monitoring and observability

5.5 Microsegmentation Model

The microsegmentation model in SecureHealthCare is implemented as a hybrid of static network segmentation and dynamic identity-aware service isolation. Rather than placing all healthcare services on one flat network, the prototype separates them across multiple Docker bridge networks, namely **edge-net**, **control-net**, **backend-segment-net**, **backend-support-net**, **data-private**, **security-net**, and dedicated observability networks. This creates explicit trust boundaries between user-facing services, policy services, application services, data stores, and monitoring components.

Let the set of protected segments be represented as

$$\mathcal{S} = \{S_{edge}, S_{control}, S_{clinical}, S_{support}, S_{data}, S_{security}, S_{observe}\}.$$

Each service v is assigned only to the segment or segments required for its function. For example, the frontend and external API gateway operate at the edge, the SDP and SPA controllers operate in the control plane, the backend and internal gateway operate in the protected clinical segment, the database is confined to the data-private segment, and the encryption service is restricted to the backend

support segment.

Communication is therefore not defined by implicit network reachability, but by an allowed communication relation $A(i, j)$ over services:

$$A(i, j) = \begin{cases} 1, & \text{if } (i, j) \in E_{allow} \text{ and policy conditions are satisfied} \\ 0, & \text{otherwise} \end{cases}$$

where E_{allow} contains only explicitly permitted paths. In the implemented prototype, the principal allowed paths are:

1. frontend $\rightarrow$ API gateway
2. API gateway $\rightarrow$ backend internal gateway
3. backend internal gateway $\rightarrow$ backend service
4. backend $\rightarrow$ database
5. backend $\rightarrow$ encryption service
6. protected services $\rightarrow$ telemetry collector

All other direct paths, especially edge-to-database, edge-to-backend, IAM-to-backend, and control-plane-to-data paths, are denied by default. This default-deny structure directly limits lateral movement inside the hospital environment.

For a request $r = (u, c, p, m, s)$, where u is the authenticated user, c is the client device, p is the requested path, m is the HTTP method, and s is the destination service, access is granted only if all Zero Trust conditions hold:

$$Permit(r) = Auth(u) \wedge MFA(u) \wedge Policy(u, p, m) \wedge GatewayPath(s) \wedge \neg Isolated(S_s)$$

where S_s denotes the segment containing the destination service. Thus, even if

a user has valid credentials, the request is denied when the target segment has been placed into an isolated state.

This model is reinforced by the Software-Defined Perimeter workflow. Protected requests do not reach backend services directly. They must first pass through the external API gateway, then the SDP policy engine, and finally the backend internal gateway before reaching the application. The SPA controller issues temporary admissions and grants only for validated clients and authorized users. As a result, the protected clinical segment remains hidden from unauthorized clients and is reachable only through approved gateway chains.

Microsegmentation is also adaptive during incident response. When a critical alert such as exfiltration, ransomware indication, or correlated high-confidence anomaly is generated, the response layer revokes active grants and the access controller marks the affected segment as isolated. Conceptually, for a segment S_k ,

$$Isolated(S_k, t^+) = 1$$

which causes subsequent grant requests toward that segment to be denied until recovery is performed. Audit logs from the implemented system show this behavior through `segment_isolated`, `grant_denied`, and `segment_released` events recorded for the `backend-clinical-segment`.

Therefore, the proposed microsegmentation model is not limited to simple host blocking. It combines network-level separation, gateway-mediated application reachability, identity-aware authorization, and response-driven segment isolation. This design is particularly suitable for healthcare environments because it protects critical assets such as patient databases and encryption services while still allowing legitimate clinical workflows to proceed through narrowly defined communication paths.

Chapter 6

Results and Discussion

This chapter evaluates the implemented SecureHealthCare system under realistic healthcare environment conditions, focusing on access control, event generation, telemetry collection, rule-based detection, machine learning-based behavioral analysis, centralized response, and monitoring observability.

6.1 Validation of SDP Access Control

The SecureHealthCare SDP framework successfully demonstrated identity-aware access control and dynamic protection of the hospital backend during the demo. Validation of the access path showed that protected services were not exposed directly to clients; instead, access was mediated through the API Gateway, SDP policy evaluation, SPA-based admission, and the protected backend service registration flow.

Functional verification confirmed that the SDP controller correctly enforced role-based access policies for protected hospital APIs. Authorized requests were allowed only when the user role and requested resource matched the configured policy, while unauthorized requests were denied with explicit policy reasons. During the demo, valid role-path combinations were permitted, whereas forbidden combinations were rejected according to the configured access rules. This confirmed that the API Gateway and SDP controller jointly enforced the intended

Zero Trust restrictions.

```
=====
2026-04-21T22:13:01.335Z POLICY_DENIED
Category : access
Outcome  : denied
Summary  : Policy denied hospital-backend-app for accountant@hospital.com (accountant)
Actor    : accountant@hospital.com (accountant)
Target   : hospital-backend-app
Reason   : role_denied
Details  :
- serviceId: "hospital-backend-app"
- pathname:  "/api/audit"
- method:   "GET"
- reason:    "role_denied"
- user:     {"email":"accountant@hospital.com","role":"accountant"}
```

Figure 6.1: Role-based policy enforcement in the SecureHealthCare SDP demo, showing allow and deny decisions for protected hospital resources

Unauthenticated access attempts were also correctly blocked. Requests sent to protected endpoints without the required authentication context were rejected by the gateway and policy chain, confirming that backend resources were not reachable without prior identity validation. This shows that the SDP framework prevents direct access to protected hospital services when a valid token or access context is absent.

```
2026-04-21T22:13:08.028Z GRANT_DENIED
Category : access
Outcome  : denied
Summary  : Grant denied for hospital-backend-app on n/a because missing_spa_admission
Actor    : admin-laptop-01
Target   : hospital-backend-app
Reason   : missing_spa_admission
Details  :
- clientId: "admin-laptop-01"
- serviceId: "hospital-backend-app"
- reason:    "missing_spa_admission"
```

Figure 6.2: SecureHealthCare SDP demo showing rejection of access to a protected endpoint when valid authentication context is not presented

The demo further showed that protected access depended on SPA-based admission. The audit trail recorded the creation and repeated validation of a temporary SPA admission for the client `admin-laptop-01` before protected access was granted. This confirms that the SecureHealthCare architecture does not rely

on static reachability alone; instead, client admission is explicitly validated before requests are forwarded to protected services.

```
=====
2026-04-21T22:13:00.649Z SPA_ADMISSION_VALIDATED
Category : access
Outcome  : allowed
Summary  : SPA admission validated for admin-laptop-01
Actor    : admin-laptop-01
Details  :
- clientId: "admin-laptop-01"
- clientCertCn: "admin-laptop-01"
- sourceIp: "172.28.0.1"
- admittedAt: "2026-04-21T22:12:55.045Z"
- expiresAt: "2026-04-21T22:17:55.045Z"
```

Figure 6.3: SPA-based admission in the SecureHealthCare demo, showing successful client validation before protected service access is granted

Finally, the SDP demo validated dynamic microsegmentation during an incident. When the protected backend segment was marked as isolated, the `backend-clinical-segment` was moved into a protected state, and subsequent access requests were denied. This shows that the SecureHealthCare framework not only controls initial access, but can also dynamically revoke or block access to protected services during attack conditions, thereby limiting lateral movement and containing the affected segment.

```
=====
2026-04-21T22:13:04.610Z SEGMENT_ISOLATED
Category : segmentation
Outcome  : isolated
Summary  : Segment backend-clinical-segment isolated for hospital-backend-app because demo_segment_isolation
Target   : backend-clinical-segment
Reason   : demo_segment_isolation
Details  :
- segmentId: "backend-clinical-segment"
- serviceId: "hospital-backend-app"
- reason: "demo_segment_isolation"
- isolatedAt: "2026-04-21T22:13:04.587Z"
- affectedServices: ["hospital-backend-app"]
```

Figure 6.4: Dynamic isolation of the `backend-clinical-segment` in the SecureHealthCare SDP demo following a security event

6.2 Validation of Event Generation and Security-Relevant Activity

Access to the hospital portal follows a multi-stage authentication workflow in which the credential-based login is followed by a six-digit TOTP verification step.

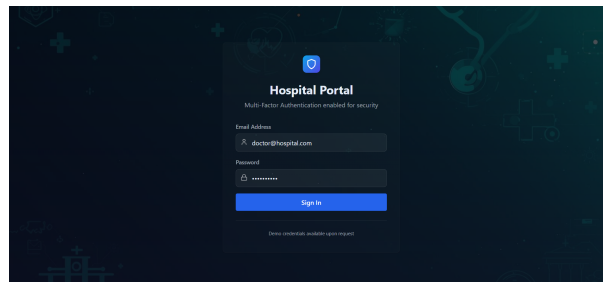


Figure 6.5: Login interface of the hospital portal used to initiate authenticated access

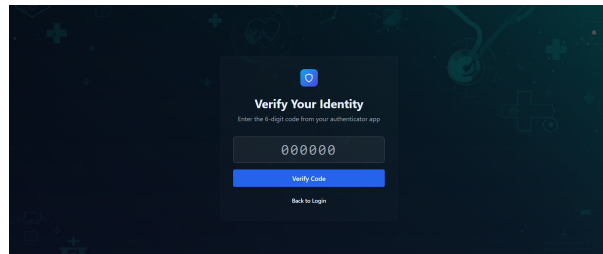


Figure 6.6: MFA verification stage requiring a six-digit authenticator code before access is granted

This was further validated through frontend network and console evidence. The initial login response requires MFA, while the verification response returns a successful session together with an access token, a refresh token, and an authenticated user context. This confirms that session credentials are issued only after successful completion of the MFA step.

```

Attempting login to /api/login                                     auth.ts:58
Login response status: 200                                       auth.ts:64
Login response: Object {                                         auth.ts:66
  message: "MFA required",
  mfaRequired: true,
  success: true,
  [[Prototype]]: Object
}
Attempting MFA verify to /api/mfa/verify                         auth.ts:77
MFA verify response status: 200                                   auth.ts:83
MFA verify response:                                             auth.ts:85
Object {
  refreshToken: "469246985a3cbf6e0629f9cc25e0678b7a36592ee34c3293ab4ff74ae22dcf51",
  success: true,
  token: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJlbWV3RvckBob3NwaXRhbC5jb201LCJyb2x1Ijo1ZG9jdG9yIiwiaWF0IjoxNzIzOTgwLjEhHA10JF",
  user: {email: 'doctor@hospital.com', name: 'Dr. John Smith', role: 'doctor', permissions: [-]}
  [[Prototype]]: Object
}

```

Figure 6.7: Console evidence showing MFA enforcement and issuance of access and refresh tokens after successful verification

Role-constrained event generation was also observed during runtime. A structured `ROLE_ACCESS_VIOLATION` telemetry record was generated when a user attempted to access a resource outside their assigned role. The event preserves user identity, role, HTTP method, requested path, and response status.

```

PS D:\FINAL_YEAR\fyppmodif-main> (Invoke-RestMethod "http://localhost:9090/telemetry").recentTelemetry |
>> Where-Object { $_.eventType -eq "ROLE_ACCESS_VIOLATION" } |
>> Select-Object hostId, ts, eventType, source, security |
>> ConvertTo-Json -Depth 10
{
  "hostId": "hospital-backend",
  "ts": "2026-03-13T17:52:45.885Z",
  "eventType": "ROLE_ACCESS_VIOLATION",
  "source": "manual-proof",
  "security": {
    "userEmail": "accountant@hospital.com",
    "userRole": "accountant",
    "method": "GET",
    "path": "/api/labs/dashboard",
    "statusCode": 403,
    "ip": "172.21.0.156"
  }
}

```

Figure 6.8: Structured `ROLE_ACCESS_VIOLATION` telemetry generated when a user attempted to access a resource outside the assigned role

6.3 Validation of Centralized Telemetry and Data Collection

The central collector stored records from multiple hospital components, including workstation agents and service-level producers, together with role context, iden-

tity fields, source information, and collector-assigned timestamps. This confirms that telemetry was not retained in isolated local logs, but was normalized and centralized for downstream analysis.

```
[
  {
    "hostId": "admin-workstation",
    "userRole": "admin",
    "userEmail": "admin@hospital.com",
    "source": "edr-agent",
    "receivedAt": "2026-03-13T19:11:44.273Z"
  },
  {
    "hostId": "nurse-workstation",
    "userRole": "nurse",
    "userEmail": "nurse@hospital.com",
    "source": "edr-agent",
    "receivedAt": "2026-03-13T19:11:44.775Z"
  },
  {
    "hostId": "hospital-iam",
    "userRole": "unknown",
    "userEmail": "unknown",
    "source": "edr-agent",
    "receivedAt": "2026-03-13T19:11:44.810Z"
  },
  {
    "hostId": "hospital-iam",
    "userRole": null,
    "userEmail": null,
    "source": "edr-role-middleware",
    "receivedAt": "2026-03-13T19:11:44.987Z"
  }
]
```

Figure 6.9: Filtered centralized telemetry output showing multiple producers forwarding structured records into a single monitoring collector

Given below is host-level telemetry as shown in the figure . The record pre-

serves the host identity, user role, email, source, and collector timestamp, while also indicating the presence of process activity, network activity, and database-related monitoring. This confirms that the runtime context of endpoint-style telemetry agents is incorporated into the centralized stream.

```
{
  "hostId": "accountant-workstation",
  "userRole": "accountant",
  "userEmail": "accountant@hospital.com",
  "source": "edr-agent",
  "receivedAt": "2026-03-13T19:13:39.348Z",
  "processCount": 3,
  "networkCount": 14,
  "hasDbActivity": true,
  "dbMonitorRole": "accountant"
}
```

Figure 6.10: Compact host-level telemetry record from the centralized collector, showing runtime context contributed by a workstation agent

Structured security telemetry was also present within the same collector output. Figure 6.11 shows a `ROLE_ACCESS_VIOLATION` event preserved in normalized form with host identity, timestamp, user identity, role, HTTP method, request path, and response status. This demonstrates that the centralized telemetry layer includes not only host-level runtime activity but also application-layer security events required for correlation and later detection.

```
{
  "hostId": "hospital-backend",
  "ts": "2026-03-13T19:15:29.442Z",
  "eventType": "ROLE_ACCESS_VIOLATION",
  "userEmail": "accountant@hospital.com",
  "userRole": "accountant",
  "method": "GET",
  "path": "/api/labs/dashboard",
  "statusCode": 403
}
```

Figure 6.11: Structured `ROLE_ACCESS_VIOLATION` telemetry within the centralized collector, illustrating normalization of role-based security events for downstream analysis.

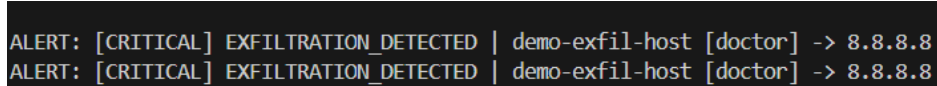
6.4 Validation of Rule-Based Traffic and Security Analysis

To validate rule-based exfiltration detection, a telemetry record containing a suspicious IP address was introduced into the centralized stream. The record preserved host identity, user context, source information, and network fields, including a destination address of `8.8.8.8`, which matches one of the suspicious IPs hardcoded in the analyzer.

```
{
  "hostId": "demo-exfil-host",
  "ts": "2026-03-13T19:32:05.159Z",
  "userRole": "doctor",
  "userEmail": "doctor@hospital.com",
  "source": "manual-proof",
  "net": {
    "src": "172.20.0.20",
    "dst": "8.8.8.8",
    "port": 443
  }
}
```

Figure 6.12: Injected telemetry record used to simulate suspicious outbound traffic toward a destination IP matched by the exfiltration rule

The traffic analyzer classified the telemetry item as `EXFILTRATION_DETECTED` with `CRITICAL` severity, demonstrating that the rule engine successfully matched the suspicious network destination and converted the telemetry into a structured alert.



```
ALERT: [CRITICAL] EXFILTRATION_DETECTED | demo-exfil-host [doctor] -> 8.8.8.8
ALERT: [CRITICAL] EXFILTRATION_DETECTED | demo-exfil-host [doctor] -> 8.8.8.8
```

Figure 6.13: Traffic analyzer output showing deterministic rule-based detection of exfiltration with `CRITICAL` severity

These results confirm that rule-based traffic and security analysis functioned as an active detection layer capable of transforming centralized telemetry into actionable security alerts.

6.5 Validation of Machine Learning-Based Behavioural Analysis

Using the CSE-CIC-IDS2018 dataset, the training pipeline processed 6,659,532 available records and applied controlled subsampling to construct a working dataset of 398,656 samples, consisting of 319,368 benign instances and 79,288 attack instances. During preprocessing, the pipeline removed constant features, applied logarithmic transformations to skewed attributes, pruned highly correlated variables, and selected the 35 most important features for the final model construction.

The Random Forest model was trained on 279,059 samples and evaluated on stratified validation and test sets. Threshold optimization identified **0.14** as the most suitable decision cutoff point for distinguishing benign from malicious traffic, improving attack recall while maintaining acceptable precision and false positive behavior.

Table 6.1: Random Forest Model — Final Evaluation Results

Metric	Value
Recall	0.9527
Precision	0.7289
F1-Score	0.8259
ROC-AUC	0.9872
False Positive Rate	0.0880
True Positives (TP)	11,331
True Negatives (TN)	43,692
False Positives (FP)	4,214
False Negatives (FN)	562

```

Recall:    0.9527
Precision: 0.7289
F1-score:  0.8259
ROC-AUC:   0.9872
FPR:       0.0880
Confusion: TP=11,331 TN=43,692 FP=4,214 FN=562

```

Figure 6.14: Final evaluation results of the Random Forest model used for machine learning-based behavioral analysis

The training and evaluation results validate the effectiveness of the ML-based behavioral analysis stage in distinguishing benign from malicious patterns within the intrusion dataset. The high recall of 95.27% demonstrates that the model is effective in identifying attacks, while the ROC-AUC of 0.9872 confirms strong overall discriminative performance. The trained model was thereby considered suitable for integration into the system's anomaly-detection layer.

6.6 Validation of Centralized Response and Enforcement

Different types of alert resulted in different response actions. A `ROLE_ACCESS_VIOLATION` produced a `revoke_tokens` action with `HIGH` severity, while an `EXFILTRATION_DETECTED` event produced an `isolate` action with `CRITICAL` severity. This confirms that enforcement was selected according to both event type and severity rather than being applied uniformly.

```
[
  {
    "hostId": "demo-exfil-host",
    "action": "isolate",
    "reason": "EXFILTRATION_DETECTED",
    "severity": "CRITICAL",
    "ts": "2026-03-13T19:36:14.228Z"
  }
]
```

Figure 6.15: Response-controller action history showing severity-based enforcement actions, including `revoke_tokens` for role violation and `isolate` for critical exfiltration detection

The response history also showed that enforcement actions persisted together with the host identity, trigger reason, severity, and timestamp. Rollback support was available through a recorded `revert_isolation` action for the affected host. These results confirm that the system has gone beyond passive detection to centralized, auditable, and reversible response enforcement.

```
[
  {
    "hostId": "demo-exfil-host",
    "action": "isolate",
    "reason": "EXFILTRATION_DETECTED",
    "ts": "2026-03-13T19:36:14.228Z"
  },
  {
    "hostId": "demo-exfil-host",
    "action": "revert_isolation",
    "reason": "manual_recovery",
    "ts": "2026-03-13T19:39:02.517Z"
  }
]
```

Figure 6.16: Recorded isolation and revert_isolation actions demonstrating that centralized enforcement actions are auditable and reversible

Table 6.2 summarizes the response actions triggered for each alert type.

Table 6.2: Response Actions by Alert Type and Severity

Alert Type	Severity	Response Action	IAM Action
ROLE_ACCESS_VIOLATION	HIGH	restrict	Token revocation
EXFILTRATION_DETECTED	CRITICAL	isolate	Token revocation + IP block
RANSOMWARE_INDICATOR	CRITICAL	isolate	Token revocation + IP block
BRUTE_FORCE_ATTEMPT	HIGH	restrict	Token revocation
ML_ANOMALY (low confidence)	LOW	log_only	None

6.7 Monitoring and Observability

The implemented SecureHealthCare architecture provided runtime observability through a Grafana dashboard built on Prometheus-collected metrics. Unlike a generic infrastructure dashboard, the deployed view exposed the live state of the project’s SDP and microsegmentation model at the segment level. The dashboard

reported the number of observed segments, the number of actively isolated segments, the number of registered gateways, total telemetry events, segment-wise activity volume, suspicious connections, and temporal telemetry trends.

At the time of validation, the dashboard showed **5 observed segments**, **1 active isolated segment**, **2 registered gateways**, and **370 telemetry events**. This confirms that the monitoring stack was not limited to raw service health checks, but was capturing security-relevant state from the SecureHealthCare segmentation and access-control workflow. In particular, the dashboard reflects the project’s protected network structure, where multiple segments are continuously monitored, and isolation state is made visible during enforcement.

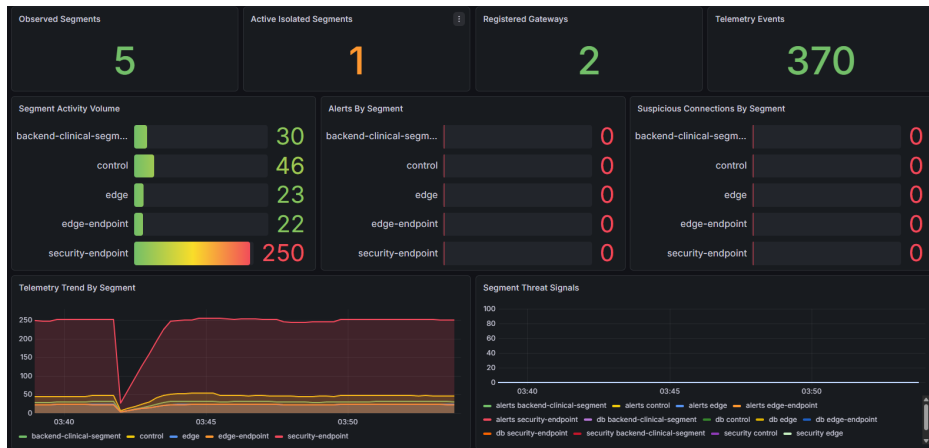


Figure 6.17: Grafana dashboard for the SecureHealthCare SDP and microsegmentation demo, showing observed segments, active segment isolation, registered gateways, telemetry volume, and segment-wise activity trends

The segment activity panels further showed that telemetry was distributed across the protected environment rather than originating from a single source. The **security-endpoint** segment contributed the highest activity volume, while the **control**, **backend-clinical-segment**, **edge**, and **edge-endpoint** segments also generated measurable telemetry. In contrast, both the *Alerts by Segment* and *Suspicious Connections by Segment* panels showed zero values across all segments

in the captured state, indicating that although telemetry and segmentation activity were being actively monitored, no suspicious connection events were present at the moment represented by the dashboard.

The telemetry trend panel also confirmed continuous observation over time across multiple segments, with the `security-endpoint` segment contributing the largest share of telemetry during the displayed interval. Together, these results show that Prometheus-based collection and Grafana-based visualization worked as an integrated observability layer for SecureHealthCare, enabling continuous monitoring of segmentation state, service registration, isolation status, and segment-level activity.

Table 6.3 summarizes the key metrics observed on the Grafana dashboard during testing.

Table 6.3: Key Metrics from the SecureHealthCare Grafana Dashboard

Metric	Observed Value
Observed Segments	5
Active Isolated Segments	1
Registered Gateways	2
Telemetry Events	370
Backend Clinical Segment Activity	30
Control Segment Activity	46
Edge Segment Activity	23
Edge Endpoint Activity	22
Security Endpoint Activity	250

6.8 Summary of Results

The evaluation confirms that the SecureHealthCare prototype successfully implemented all major capabilities of the proposed Zero Trust framework. Table 6.4 provides a consolidated summary of the validation outcomes.

Table 6.4: Summary of Validation Results

Validation Area	Mechanism	Outcome
SDP Access Control	Role-based policy enforcement	Correctly allowed/denied per role
Multi-Factor Authentication	TOTP + bcrypt + JWT	Tokens issued only after MFA
Event Generation	Role middleware + file operations	ROLE_ACCESS_VIOLATION captured
Centralized Telemetry	Multi-producer JSON stream	Unified monitoring collector
Rule-Based Detection	Hardcoded threat signatures	EXFILTRATION detected, CRITICAL alert
ML Behavioural Analysis	Random Forest (CSE-CIC-IDS2018)	Recall 95.27%, ROC-AUC 0.9872
Automated Response	Severity-based enforcement	Isolate, revoke, restrict actions
Monitoring (Grafana)	Prometheus + Grafana dashboards	Real-time security metrics visible

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project offered SecureHealthCare as a zero trust architecture approach specifically aimed at healthcare with IAM, SDP-based access controls, EDR, telemetry, rules for detection, machine learning for behavioral analysis, and response automation. In order to achieve the objective, the project used an architecture design divided into two phases: Phase I for authentication, authorization, and secure healthcare access, and Phase II for monitoring and centralizing security operations.

The project successfully proved the possibility of integrating an identity-based access control solution along with collecting security telemetry, detecting security policy violations and suspicious outgoing connections, and converting them into security procedures such as token revocation, access restrictions, and isolating the affected host. The attack recall for the Random Forest algorithm was 95.27%, showing how well machine learning and rule-based detection complement each other in finding suspicious activities.

The results of the study demonstrate that SecureHealthCare establishes a real-world foundation that enables healthcare organizations to protect their systems through identity-based access control, telemetry-driven system oversight, and automated security containment processes.

7.2 Future Work

Future work will aim to enhance the machine learning system through improved explainability, threshold calibration, and runtime validation, which will help reduce false positive errors while maintaining effective detection capabilities. The framework needs further testing through actual healthcare scenarios that require monitoring of IoMT devices and handling of extensive healthcare operations.

The upcoming enhancements will include the following features:

- **Dynamic risk assessment:** Developing continuously updated risk scores for users and devices based on real-time behavioral signals.
- **Compliance-based audit logs:** Generating structured audit reports aligned with HIPAA and other regulatory requirements.
- **Recovery procedures:** Implementing automated recovery workflows that restore isolated services and revoked tokens after threat containment is confirmed.
- **Integration of IoMT devices:** Extending the deployment of EDR agents to embedded medical devices of the IoT with a lightweight telemetry collection.
- **Explainability in ML:** Integrating SHAP (SHapley Additive exPlanations) to provide human-readable explanations for anomaly detections.

Through these improvements, the system will become more adaptable, forensic-ready, and deployable for healthcare environments.

Appendix A

Supplementary Material

A.1 Docker Compose Service Overview

The SecureHealthCare prototype was deployed using Docker Compose. The following services were included in the containerized environment:

Table A.1: Docker Compose Services in SecureHealthCare Prototype

Service	Description
frontend	React-based hospital portal serving the user interface
backend	Node.js REST API providing healthcare data endpoints
iam-service	Identity and Access Management service for authentication and token management
api-gateway	Reverse proxy that validates tokens and routes requests
sdp-controller	Enforces role-based access policies via SDP
encryption-service	AES-256-GCM encryption and decryption service
monitoring-service	Telemetry collector and Prometheus metrics exporter

Continued on next page...

Service	Description
response-controller	Automated response engine for isolation and token revocation
ml-engine	Python-based Random Forest inference service
postgres	PostgreSQL database storing encrypted patient records
prometheus	Time-series metrics collection
grafana	Dashboard visualization for operational and security metrics

A.2 CSE-CIC-IDS2018 Dataset Summary

The CSE-CIC-IDS2018 dataset was used for training and evaluating the ML-based behavioral analysis component. Table A.2 summarizes the dataset statistics used in this project.

Table A.2: CSE-CIC-IDS2018 Dataset Statistics

Parameter	Value
Total available records	6,659,532
Working dataset (after subsampling)	398,656
Benign instances	319,368
Attack instances	79,288
Training samples	279,059
Features selected (after engineering)	35
Decision threshold (optimized)	0.14

A.3 User Roles and Permissions Summary

The system defined eight hospital user roles with distinct permissions. Table A.3 summarizes the role-to-resource access mapping.

Table A.3: Hospital User Roles and Permitted Resource Endpoints

Role	Permitted Endpoints
Admin	All endpoints (bypass mechanism)
Doctor	Patient records, appointments, laboratory results, reports
Nurse	Patient records, appointments, clinical notes
Receptionist	Appointments, patient registration
Lab Technician	Laboratory results, test orders
Pharmacist	Prescriptions, medication records
Accountant	Billing, limited patient summary
Patient	Own records, appointment booking

Bibliography

- [1] P. Bajpayi, S. Sharma, and M. S. Gaur, “AI Driven IoT Healthcare Devices Security Vulnerability Management,” in *2024 2nd International Conference on Disruptive Technologies (ICDT)*, Greater Noida, India, 2024, pp. 366–373. doi: 10.1109/ICDT61202.2024.10488939. 1
- [2] A. Khan and I. Sharma, “Machine Learning-Based Methodology for Preventing Ransomware Attacks on Healthcare Sector,” in *2023 International Conference on Research Methodologies in Knowledge Management, Artificial Intelligence and Telecommunication Engineering (RMKMATE)*, Chennai, India, 2023, pp. 1–5. doi: 10.1109/RMKMATE59243.2023.10368971. 1
- [3] N. Alsuwaidi, A. Alshamsi, S. Alshamsi, and S. Zeadally, “The Transformative Impact of Zero-Trust Architecture on Healthcare Security,” in *2024 2nd International Conference on Cyber Resilience (ICCR)*, Dubai, United Arab Emirates, 2024, pp. 1–6. doi: 10.1109/ICCR61005.2024.10532794.
- [4] S. Munasinghe, N. Piyaarathna, E. Wijerathne, U. Jayasinghe, and S. Nimal, “Machine learning based zero trust architecture for secure networking,” in *2023 IEEE 17th International Conference on Industrial and Information Systems (ICIIS)*, Peradeniya, Sri Lanka, 2023, pp. 1–6. doi: 10.1109/ICIIS58898.2023.10253610. 2
- [5] B. C. Mocanu, R. Stoleriu, A. E. Mocanu, C. Negru, E. G. Dragotoiu, M. A. Moisescu, and F. Pop, “NextEDR - Next generation agent-based EDR systems for cybersecurity threats,” in *2024 32nd Euromicro International*

- Conference on Parallel, Distributed and Network-Based Processing (PDP)*, Dublin, Ireland, 2024, pp. 183–190. doi: 10.1109/PDP62837.2024.00035. 2, 6, 8
- [6] N. Sheikh, M. Pawar, and V. Lawrence, “Zero Trust Using Network Micro Segmentation,” in *2021 IEEE International Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, Vancouver, BC, Canada, 2021, pp. 1–6. doi: 10.1109/INFOCOMWKSHPS51825.2021.9484462. 2, 6
- [7] K. Geetha, A. G. Sreedevi, and A. R. Chadha, “Unraveling IoT Network Security with Snort for Robust Intrusion Detection and Prevention,” in *2024 IEEE International Conference on Contemporary Computing and Communications (InC4)*, Bangalore, India, 2024, pp. 1–6. doi: 10.1109/InC460750.2024.10649075. 7, 8
- [8] K. Kandasamy, S. Srinivas, K. Achuthan, and V. P. Rangan, “Digital Healthcare - Cyberattacks in Asian Organizations: An Analysis of Vulnerabilities, Risks, NIST Perspectives, and Recommendations,” *IEEE Access*, vol. 10, pp. 12345–12364, 2022. doi: 10.1109/ACCESS.2022.3145372. 5
- [9] S. Alzahrani, Y. Xiao, S. Asiri, J. Zheng, and T. Li, “A Survey of Ransomware Detection Methods,” *IEEE Access*, vol. 13, pp. 57943–57982, 2025. doi: 10.1109/ACCESS.2025.3556187. 5
- [10] D. Abhiram, R. Harish, and K. Praveen, “Zero-Trust Security Implementation Using SDP over VPN,” in *Inventive Communication and Computational Technologies*, Singapore: Springer, 2022, pp. 267–276. 6, 8
- [11] C. Zanasi, M. Marchetti, and M. Colajanni, “Cybersecurity Domains: A design pattern for creating Zero Trust Architectures through microsegmentation,” in *2024 IEEE Conference on Dependable, Autonomic and Secure Com-*

- puting (DASC)*, Boracay, Philippines, 2024, pp. 15–22. doi: 10.1109/DASC-PICom-CBDCom-CyberSciTech64126.2024.00015. 6
- [12] S. Maniath, A. Ashok, P. Poornachandran, V. G. Sujadevi, A. U. P. Sankar, and S. Jan, “Deep learning LSTM based ransomware detection,” in *2017 Recent Developments in Control, Automation Power Engineering (RDCAPE)*, Noida, India, 2017, pp. 442–446. doi: 10.1109/RDCAPE.2017.8358312. 6, 8
- [13] R. Vinayakumar, K. P. Soman, and P. Poornachandran, “Deep Android Malware Detection and Classification,” in *2017 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, Udupi, India, 2017, pp. 1677–1683. doi: 10.1109/ICACCI.2017.8126084. 6
- [14] S. A. Shashank, V. K. Prajapati, P. V. Manitha, and M. Nithya, “IoT-driven Health Monitoring System Coupled with Blockchain,” in *2023 Fourth International Conference on Smart Technologies in Computing, Electrical and Electronics (ICSTCEE)*, Bengaluru, India, 2023, pp. 1–5. doi: 10.1109/ICSTCEE60504.2023.10585241. 7
- [15] M. S. Urmila, B. Hariharan, and R. Prabha, “A comparative study of blockchain applications for enhancing internet of things security,” in *2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, Kanpur, India, 2019, pp. 1–7. doi: 10.1109/ICCCNT45670.2019.8944446. 7